Live Demo Guide - Automated Device Compliance Management Portal

Total time: about **20 minutes** (plus questions). The start script, the Selenium run and both Ansible runs were rehearsed on this machine and timed; the Jenkins build time (about 2 min 20 s) comes from the real builds #6 and #7.

1. The night before

- [ ] Charge the laptop and bring the charger. Close other apps (Spotify, chat, mail); turn on Do Not Disturb.
- [ ] Run the start script once and read the status table (section 2). Everything must say **OK**.
- [ ] Open Jenkins (http://localhost:9090) and **sign in with your own account** - do it before the demo, never in front of her.
- [ ] Put a copy of `report/DCMP_DevOps_Report.pdf` on your phone or a USB stick (backup if the laptop fails).
- [ ] Know whether the room has internet. **It is only needed for the live Jenkins build** (it fetches the code from GitHub). Everything else runs offline. No internet: skip the live build and show the finished builds instead (section 5, part 4).

2. 15 minutes before: start everything

Open **PowerShell** in the project folder and run:

```
cd C:\Users\damal\Downloads\device-compliance-portal
powershell -ExecutionPolicy Bypass -File scripts\demo-start.ps1
```

It starts only what is not running (Docker Desktop, registry, containers, Jenkins, Tomcat) and ends with a table:

Service	URL	What it is
Jenkins	http://localhost:9090	CI/CD server
App on Tomcat	http://localhost:8081/dcmp/	step 8 deployment
App in Docker	http://localhost:8092/	steps 11-12 container (clean demo data)
App on Ansible node	http://localhost:8093/	steps 13-14 provisioned server
Registry	http://localhost:5000/v2/_catalog	Docker images

A cold start takes about 30 seconds if Docker is already running; allow 2-3 minutes if Docker Desktop must start.

Browser tabs to open, in this order (so you can click through without typing):

1. GitHub repo - https://github.com/Athdam05/device-compliance-portal
2. App (Docker) - http://localhost:8092/
3. Jenkins pipeline - http://localhost:9090/job/dcmp-pipeline/
4. The report PDF (as backup and for the checklist)

Two terminals: PowerShell (Docker commands) and **Git Bash** (Ansible), both in the project folder, with a large font (zoom with Ctrl+scroll). Zoom the browser to 125%.

Logins - app: `admin / admin123`, `auditor / auditor123`, `tech / tech123` (demo accounts; passwords are stored as BCrypt hashes). Jenkins: your own.

3. The story to tell (60 seconds)

> *"The project is a Device Compliance Management Portal. Devices are registered, checked against compliance rules - antivirus, disk encryption, firewall - and moved through a role-based workflow. The assignment is about the DevOps pipeline around it, so I will show the path of one change: from Git, through Jenkins build and tests, to a Docker container, and onto a server configured by Ansible - including what happens when a release is bad."*

4. Demo flow and timing

#	Part	Minutes	Steps covered
1	Problem, plan and repository	3	1-6
2	The application	3	3, 5, 6
3	Jenkins: live build + the failed/fixed story	6	7, 8, 9, 10
4	Docker and the registry	2	11, 12
5	Ansible: provisioning, idempotency, rollback	4	13, 14
6	Wrap-up with the report	1	all

5. What to do and say, part by part

Part 1 - Problem, plan, repository (3 min) - GitHub tab

- **README**: problem, stack, branch policy. *"Main is stable, develop is integration, every change comes through a pull request."*
- **Issues** (4 user stories) - planning (steps 1-2). Report pages 3-10 have the stakeholders, backlog, Kanban boards, DoD and architecture diagrams.
- **Pull requests -> #4**: shows a **review comment** on `WorkflowService.java`, merged. Then open commit `bb8f57f`: *"This is the merge conflict I created on purpose in `list.html`, and how I resolved it."*
- **Tags**: `v1.0.0` (MVP) and `v1.1.0` (complete project).

Part 2 - The application (3 min) - tab http://localhost:8092/

1. Log in as **tech / tech123** -> the device list. Click **Add device**, submit it **empty** -> validation errors. Then register `Demo-Laptop`, owner `Your name`, serial `DEMO-` + today's time (serial must be unique), tick only *Antivirus* -> it is **NON_COMPLIANT** (all three controls are needed).
2. Open the device -> **Move to UNDER_REVIEW** -> **Move to REMEDIATED**. The history table records who and when. Point out the technician cannot close it.
3. **Logout**, log in as **auditor / auditor123** -> open the device -> the auditor can **close** it or reopen it to UNDER_REVIEW. Click **Move to CLOSED**.
4. **Dashboard** -> totals update. *"Three roles, rules enforced on the server, every transition recorded."*

Part 3 - Jenkins (6 min) - tab http://localhost:9090/job/dcmp-pipeline/

1. Click **Build with Parameters** -> leave `dev` / `dcmp` -> **Build**. It takes **about 2 min 20 s** (needs internet).

2. While it runs, open the **Jenkinsfile** on GitHub and walk through the stages: *Checkout -> Build (11 unit tests) -> Package -> **Selenium Tests** -> Deploy to Tomcat -> Verify -> Docker Build -> Push -> Deploy*. Mention the parameter `DEPLOY_ENV` selects the Spring profile (this is the "parameterised environment setting" of step 8).
3. Open **Stages** (left menu). Show the history: **#3 is red at Selenium and Deploy is skipped** - "*I introduced a defect on purpose: the compliance rule used OR instead of AND. All 11 unit tests still passed, but the browser test caught it, so the broken version never reached a server.*" Open #3 -> **Tests** (1 failure) and the archived failure screenshot. Then **#4** (green) after the fix commit `a1cfb33`.
4. Show **#7**: "*Started by an SCM change*" - a Git commit started this build by itself (steps 7 and 12).
5. Show the live build finishing green. Open the app on Tomcat (http://localhost:8081/dcmp/).
6. **No internet?** Skip step 1; show builds #4 and #7 instead.

Selenium (optional, 36 s) - in PowerShell:

```
powershell -ExecutionPolicy Bypass -File scripts\demo-selenium.ps1 -Visible
```

A real Edge window opens and runs the 5 user journeys (6 tests). Drop `-Visible` for a silent run.

Part 4 - Docker and registry (2 min) - PowerShell

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
docker image ls localhost:5000/dcmp
curl.exe -s http://localhost:5000/v2/dcmp/tags/list
docker logs --tail 5 dcmp-app
docker restart dcmp-app
```

"*Jenkins builds a versioned image `dcmp:1.0.N`, pushes it to a local registry, pulls it back and runs a fresh container. The image carries the Git commit as a label, so any container can be traced to its code.*" After `restart`, wait about 30 s, then `curl.exe -s http://localhost:8092/actuator/health` -> `{"status":"UP"}`. The full lifecycle log is in report step 11.

Part 5 - Ansible (4 min) - Git Bash

```
cd /c/Users/damal/Downloads/device-compliance-portal
APP_VERSION=1.0.7 bash ansible/run.sh play
```

~10 s. Point at `changed=0`: "*Idempotent - running it again changes nothing.*" Show http://localhost:8093/ -> **Release: 1.0.7**.

```
APP_VERSION=1.0.8 bash ansible/run.sh play
```

~35 s, ends with a red failure on purpose. "*Release 1.0.8 is a corrupted artefact. The play deploys it, the health check fails, and the playbook rolls back to 1.0.7 by itself.*" Find the line `Release 1.0.8 FAILED health check - rolled back to 1.0.7 (UP)`. Then:

```
curl -s localhost:8093/actuator/health
```

-> still `UP`, and http://localhost:8093/ still says **Release: 1.0.7**. Optional: `APP_VERSION=1.0.6 bash ansible/run.sh rollback` rolls back manually to the earlier release; run `APP_VERSION=1.0.7 bash ansible/run.sh play` afterwards to return to 1.0.7. The target is a fresh Ubuntu container with SSH; the first-time provisioning evidence is in report step 14.

Part 6 - Wrap-up (1 min)

Open the PDF at the checklist of any step: "*Every step in the sheet has a checklist showing where each deliverable is, with screenshots and outputs.*" Offer the repository link.

6. Questions she may ask

Question	Answer
What is the difference between CI and CD here?	CI = every commit is built and tested automatically (steps 7, 10). CD = a passing build is packaged, versioned and deployed automatically (steps 8, 12).
Why does the Selenium stage run on a different port (8082)?	The new build is tested on its own throw-away instance with an empty database *before* anything is deployed; only if tests pass does Deploy run.
What is idempotency and why does it matter?	Running the automation repeatedly gives the same result and changes nothing when the node is already correct - so it is safe to re-run, e.g. after a failure.
How does the rollback work?	Each release is in its own folder; `current` is a symlink. The play remembers the old target, switches, health-checks, and on failure re-points the link and restarts.
Why supervisor instead of systemd?	The managed node is a container, which has no systemd. On a normal VM the same template would produce a systemd unit.
Why a local registry and not Docker Hub?	No account or credentials needed; the sheet allows either.
Why does the job build a feature branch?	The pipeline job was pointed at the branch that introduced the stage; the Jenkinsfile is identical on `main`.
What are the limits of the project?	Demo users and an H2 file database (not for production); the OS-version compliance rule and custom error pages were deferred; the Kanban board is documented in the repo rather than a GitHub Project; the "bad release" is a corrupted file, not a real faulty build.

7. If something goes wrong

Problem	Fix
A URL does not load	Re-run `scripts\demo-start.ps1`; it restarts whatever is down.
Docker commands fail	Docker Desktop is not running; the script starts it (wait 2-3 min).
Jenkins asks you to sign in	Sign in with your own account. If it shows an error page, wait 30 s - it is still starting.
Live Jenkins build fails or stalls	No internet is the usual cause. Say so, and show builds #4 and #7 plus the report. Do not retry in front of her.
Ansible run errors with "No such file"	The Ansible node is missing: `bash ansible/run.sh up` then `APP_VERSION=1.0.7 bash ansible/run.sh play` (about 3-4 min with internet).
Selenium window does not open	Run without `-Visible`; the result is the same (6 passed).
Everything breaks	Open the PDF: every deliverable has screenshots from the real runs.